

UCC 122-1

Cloud Architecture

Lab 3: Designing a monolithic architecture for Digi Bank with SpringBoot

Cloud Computing Professional License

University of the Mountains

By **Engineer Willy Damtchou**

June 2026

Context

In the previous workshop, students implemented a first executable version of Digi Bank as a modular monolith based on Jakarta EE, PostgreSQL, Maven, and WildFly. The goal of this step was to transform the architectural framework developed in the initial work into a structured, coherent technical implementation that students could directly manipulate.

This workshop follows the same functional scope and pedagogical approach, but with a different application development stack. This time, the Digi Bank application is developed using Spring Boot, while retaining Maven, PostgreSQL, WAR packaging, and deployment on the WildFly application server. This choice allows students to compare two technical approaches applied to the same business problem, without altering the functional objectives of the Digi Bank project.

The pedagogical value of this workshop is twofold. On the one hand, it allows participants to consolidate their understanding of a modular monolithic architecture in a modern Java environment. On the other hand, it demonstrates that the same system can be implemented with different technical frameworks while maintaining the same requirements for modularity, persistence, API exposure, testability, and deployment.

Scenario

Digi Bank's technical team now wants a second reference implementation of the system, this time based on Spring Boot. The goal is not to change the business scope or redefine the functional areas, but to rebuild a coherent first version of the banking platform using the Spring ecosystem, including Spring Boot, Spring Web, and Spring Data JPA.

The system must remain organized around the areas already identified in the core project: customer management, account management, transaction management, compliance controls, and the application entry point. The application must use PostgreSQL for data persistence, Maven for build management, be compiled in WAR format, and then deployed on WildFly as an external runtime container, following a traditional Spring Boot application deployment approach.



Your role is to produce a first stable, executable, and structured technical version of Digi Bank in Spring Boot, directly usable in IntelliJ IDEA or Visual Studio Code. This version must allow you to create the initial business components, configure data access, expose REST endpoints, validate system behavior through unit tests, and prepare a clean foundation for subsequent workshops.

Workshop objectives

At the end of this workshop, the student should be able to:

- **A3.1:** Set up a Java development environment compatible with Spring Boot, Maven, PostgreSQL and WildFly.
- **A3.2:** Create and configure a multi-module Maven project for Digi Bank.
- **A3.3:** Structure a modular monolithic Spring Boot application around the domains shared, customer, account, transaction, compliance, index and app.
- **A3.4:** Configure PostgreSQL and WildFly to run a Spring Boot application packaged as a WAR file.
- **A3.5:** Develop JPA entities, repositories, services and REST controllers with Spring.
- **A3.6:** Produce a WAR package compatible with deployment on WildFly using `SpringBootServletInitializer`.
- **A3.7:** Write and run JUnit unit tests.
- **A3.8:** Write and execute BDD scenarios with Cucumber.
- **A3.9:** Deploy the application on WildFly and verify that it is working correctly.

Resources and prerequisites

a — Reference resources

This workshop is based on the following elements:

- The Digi Bank overarching project and its functional framework
- Workshop 2, conducted with Jakarta EE, serves here as a reference for educational continuity.

- The course on cloud architecture, monolithic architectures, and modular structuring
- The traditional deployment principles of a Spring Boot application as a WAR file in an external container, with a main class extending `SpringBootServletInitializer`
- Configuring a PostgreSQL datasource in WildFly

b — Technical prerequisites

The student must have:

- JDK 17 or compatible version
- Apache Maven
- PostgreSQL
- WildFly
- IntelliJ IDEA or Visual Studio Code with Java extensions
- Git
- a machine with a sufficient configuration to run all of these components locally.

1. Environmental preparation

1.1. Installing the tools

Install the following components:

- Java JDK 17
- Maven
- PostgreSQL
- WildFly
- IntelliJ IDEA Community or Ultimate, or Visual Studio Code with Java extensions;
- Git

1.2. Basic checks

Execute the following commands in a terminal:



```
Java -version  
  
mvn -version  
  
git --version  
  
psql --version
```

1.3. Expected Results

```
user@ca:4f:12:a0:b3:c1 ~ % Java -version  
  
mvn -version  
  
git --version  
  
psql --version  
  
Java version "17.0.16" 2025-07-15 LTS  
  
Java(TM) SE Runtime Environment (build 17.0.16+12-LTS-247)  
  
Java HotSpot(TM) 64-Bit Server VM (build 17.0.16+12-LTS-247,  
mixed fashion, (sharing))  
  
Apache Maven 3.9.11 (3e54c93a704957b63ee3494413a2b544fd3d825b)  
  
Maven home: /opt/homebrew/Cellar/maven/3.9.11/libexec  
  
Java version: 17.0.16, vendor: Oracle Corporation, runtime:  
/Library/Java/JavaVirtualMachines/jdk-17.jdk/Contents/Home  
  
Default locale: fr_FR, platform encoding: UTF-8  
  
BONE name: "mac os x", version: "26.5.1", arch: "aarch64",  
family: "mac"
```

```
git version 2.50.1 (Apple Git-155 )  
  
psql (PostgreSQL) 18.4 ( Homebrew )  
  
user@ca:4f:12:a0:b3:c1 ~ %
```

The environment must correctly recognize Java, Maven, Git, and PostgreSQL. The workstation must be ready to compile a Maven project, connect to PostgreSQL, and prepare for deployment on WildFly.

2. Configuration under IntelliJ IDEA and VS Code

2.1. Configuration under IntelliJ IDEA

1. Open IntelliJ IDEA.
2. Select New Project.
3. Select Maven.
4. Define the Java SDK 17.
5. Create a parent project named digibank-parent-spring.
6. Enable automatic import of Maven projects.
7. Verify that src/main/java is recognized as the source folder.
8. Verify that src/test/java is recognized as a test folder.
9. Install or activate the following useful plugins: Spring, Maven, JUnit, Cucumber for Java, Database Tools if available.

2.2. Configuration under VS Code

1. Install the following extensions:
 - Extension Pack for Java
 - Maven for Java
 - Language Support for Java by Red Hat
 - Debugger for Java
 - Test Runner for Java
 - Spring Boot Extension Pack
 - Cucumber / Gherkin support

- 2. Verify that JDK 17 is properly configured.
- 3. Verify that Maven is correctly detected.
- 4. Verify that JUnit tests and Gherkin files are recognized.

2.3. Expected result

The chosen IDE must automatically recognize the project's Maven structure, Spring Boot dependencies, JUnit tests, and Cucumber scenario files.

3. Creation of the PostgreSQL database

3.1. Creating the database and the user

In PostgreSQL, execute the following commands:

```
CREATE DATABASE digibank_db ;  
  
CREATE USER digibank_user WITH PASSWORD 'digibank_pwd' ;  
  
GRANT ALL PRIVILEGES ON DATABASE digibank_db TO digibank_user;
```

3.2. Connection Verification

```
psql -U digibank_user -d digibank_db -h localhost
```

3.3. Expected Result

```
psql (18.4 (Postgres.app))  
  
Kind "help" for help.  
  
postgres = # CREATE DATABASE digibank_db;  
  
CREATE DATABASE  
  
postgres = # CREATE USER digibank_user WITH PASSWORD  
'digibank_pwd' ;  
  
CREATE ROLE  
  
postgres = # GRANT ALL PRIVILEGES ON DATABASE digibank_db TO  
digibank_user;  
  
GRANT  
  
postgres = #
```

The connection to the digibankdb database must be accepted without error. This database will be used for JPA persistence of the Spring Boot application.

4. Creation of the multi-module Maven project

4.1. Project Tree Structure

Create the following structure:

```
digibank - parent - spring /  
├─ pom.xml  
├─ digibank - shared /  
|   └─ pom.xml
```

```
| └─ src /  
  
| └─ main / java / com / digibank / shared /  
  
| └─ test / java / com / digibank / shared /  
  
└─ digibank - customer /  
  | └─ pom.xml  
  |  
  | └─ src /  
  |  
  | └─ main / java / com / digibank / customer /  
  |  
  | └─ main / resources /  
  |  
  | └─ test / java / com / digibank / customer /  
  |  
  └─ digibank - account /  
    | └─ pom.xml  
    |  
    | └─ src /  
    |  
    | └─ main / java / com / digibank / account /  
    |  
    | └─ main / resources /  
    |  
    | └─ test / java / com / digibank / account /  
    |  
    └─ digibank - transaction /  
      | └─ pom.xml  
      |  
      | └─ src /  
      |  
      | └─ main / java / com / digibank / transaction /  
      |  
      | └─ main / resources /  
      |  
      | └─ test / java / com / digibank / transaction /
```

```
├─ digibank - compliance /
|   ├─ pom.xml
|   └─ src /
|       ├─ main / java / com / digibank / compliance /
|       ├─ main / resources /
|       └─ test / java / com / digibank / compliance /
├─ digibank - index /
|   ├─ pom.xml
|   └─ src /
|       ├─ main / java / com / digibank / index /
|       ├─ main / resources / templates /
|       └─ test / java / com / digibank / index /
└─ digibank - app /
    ├─ pom.xml
    └─ src /
        ├─ main / java / com / digibank / app /
        ├─ main / resources /
        ├─ main / webapp / WEB-INF /
        └─ test / java / com / digibank / app /
```

This structure follows the modular organization of workshop 2, in order to maintain educational continuity while changing the application framework.

4.2. pom.xml parent

Create the file **digibank-parent-spring/pom.xml** :

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd"

    <modelVersion> 4.0.0 </modelVersion>

    <groupId> com.digibank </groupId>

    <artifactId> digibank-parent-spring </artifactId>

    <version> 1.0.0 </version>

    <packaging> pom </packaging>

    <modules>

    <module> digibank-shared </module>

    <module> digibank-customer </module>

    <module> digibank-account </module>

    <module> digibank-transaction </module>

    <module> digibank-compliance </module>

    <module> digibank-index </module>

    <module> digibank-app </module>
```

```
</modules>

<properties>
<java.version> 17 </java.version>
<spring.boot.version> 3.5.15 </spring.boot.version>
<maven.compiler.release> 17 </maven.compiler.release>
<project.build.sourceEncoding> UTF-8
</project.build.sourceEncoding>
<junit.version> 5.10.2 </junit.version>
<cucumber.version> 7.15.0 </cucumber.version>
<junit.platform.version> 1.12.2 </junit.platform.version>
</properties>

<dependencyManagement>
<dependencies>
<dependency>
<groupId> org.springframework.boot </groupId>
<artifactId> spring-boot-dependencies </artifactId>
<version> ${spring.boot.version} </version>
<type> pom </type>
<scope> import </scope>
</dependency>
</dependencies>
</dependencyManagement>
```

```
<build>

<pluginManagement>

<plugins>

<plugin>

<groupId> org.wildfly.plugins </groupId>

<artifactId> wildfly-maven-plugin </artifactId>

<version> 5.1.2.Final </version>

</plugin>


<plugin>

<groupId> org.apache.maven.plugins </groupId>

<artifactId> maven-compiler-plugin </artifactId>

<version> 3.13.0 </version>

<configuration>

<release> 17 </release>

</configuration>

</plugin>

<plugin>

<groupId> org.apache.maven.plugins </groupId>

<artifactId> maven-war-plugin </artifactId>

<version> 3.4.0 </version>

</plugin>

</plugins>
```

```
</pluginManagement>

</build>

</project>
```

5. Module creation

5.1. Shared module digibank-shared

1. pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
          xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
          xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd" >

  <parent>

    <groupId> com.digibank </groupId>

    <artifactId> digibank-parent-spring </artifactId>

    <version> 1.0.0 </version>

  </parent>

  <modelVersion> 4.0.0 </modelVersion>

  <artifactId> digibank-shared </artifactId>

  <packaging> jar </packaging>

  <dependencies>
```

```
<dependency>

<groupId> org.springframework.boot </groupId>

<artifactId> spring-boot-starter-data-jpa </artifactId>

</dependency>

</dependencies>

</project>
```

2. BaseEntity Class

```
package com.digibank.shared.model;

import jakarta.persistence. GeneratedValue ;
import jakarta.persistence.GenerationType;
import jakarta.persistence. ID ;
import jakarta.persistence. MappedSuperclass ;

@MappedSuperclass

public abstract class BaseEntity {

    @Id

    @GeneratedValue ( strategy = GenerationType.IDENTITY )

    private Long id ;
```

```
public Long getId () {  
    return id ;  
}  
}
```

3. API Response Class

```
package com.digibank.shared.dto;  
  
public class ApiResponse {  
    private String message ;  
    private boolean success ;  
  
    public ApiResponse () {  
    }  
  
    public ApiResponse (String message, boolean success) {  
        this.message = message ;  
        this.success = success ;  
    }  
  
    public String getMessage () {  
        return message ;  
    }  
}
```

```

}

    public void setMessage (String message) {

        this.message = message ;

    }

    public boolean isSuccess () {

        return success ;

    }

    public void setSuccess ( boolean success) {

        this.success = success ;

    }

}

```

5.2. digibank-customer customer module

1. Additions

Create **digibank-customer/pom.xml** with dependencies on **digibank-shared**, **spring-boot-starter-web** and **spring-boot-starter-data-jpa**.

2. Customer Entity

```

package com.digibank.customer.model;

```

```
import com.digibank.shared.model.BaseEntity;

import jakarta.persistence. Column ;

import jakarta.persistence. Entity ;

import jakarta.persistence. Table ;

@Entity

@Table (name = "customers" )

public class Customer extends BaseEntity {

    @Column (nullable = false )

    private String firstName ;

    @Column (nullable = false )

    private String lastName ;

    @Column (nullable = false , unique = true )

    private String email ;

    public Customer () {

    }

    public Customer (String firstName, String lastName, String
email) {
```

```
        this.firstName = firstName ;

        this.lastName = lastName ;

        this.email = email ;
    }

    public String getFirstName () {

        return firstName ;
    }

    public void setFirstName (String firstName) {

        this.firstName = firstName ;
    }

    public String getLastName () {

        return lastName ;
    }

    public void setLastName (String lastName) {

        this.lastName = lastName ;
    }

    public String getEmail () {

        return email ;
    }
}
```

```
public void setEmail (String email) {  
    this.email = email ;  
}  
}
```

3. Repository CustomerRepository

```
package com.digibank.customer.repository;  
  
import com.digibank.customer.model.Customer;  
import org.springframework.data.jpa.repository.JpaRepository;  
  
public interface CustomerRepository extends  
JpaRepository<Customer, Long> {  
}
```

4. Customer Service

```
package com.digibank.customer.service;  
  
import com.digibank.customer.model.Customer;
```

```
import com.digibank.customer.repository.CustomerRepository;

import org.springframework.stereotype. Service ;

import java.util.List;
import java.util.Optional;

@Service

public class CustomerService {

    private final CustomerRepository repository ;

    public CustomerService (CustomerRepository repository) {

        this . repository = repository;

    }

    public Customer createCustomer (Customer customer) {

        return repository .save(customer);

    }

    public Optional<Customer> getCustomer (Long id) {

        return repository .findById(id);

    }

    public List<Customer> getAllCustomers () {
```

```
        return repository.findAll ();
    }
}
```

5. REST Controller CustomerController

```
package com.digibank.customer.api;

import com.digibank.customer.model.Customer;
import com.digibank.customer.service.CustomerService;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping ( "/api/customers" )
public class CustomerController {

    private final CustomerService service ;

    public CustomerController (CustomerService service) {
        this.service = service ;
    }
}
```

```

    @PostMapping

    public ResponseEntity<Customer> create ( @RequestBody
Customer customer) {

        return ResponseEntity. status (HttpStatus.CREATED ).body(
service.createCustomer (customer) ) ;
    }


    @GetMapping (("/{id}") )

    public ResponseEntity<Customer> findById ( @PathVariable Long
id) {

        return service .getCustomer(id)

.map(ResponseEntity:: ok )

.orElse (ResponseEntity.notFound ( ).build());
    }


    @GetMapping

    public ResponseEntity<?> findAll () {

        return ResponseEntity. ok ( service.getAllCustomers ());
    }
}

```

5.3. digibank-account account module

1. Additions

Create this module using the same model as **digibank-customer** .

2. Account Entity

```
package com.digibank.account.model;

import com.digibank.shared.model.BaseEntity;
import jakarta.persistence. Column ;
import jakarta.persistence. Entity ;
import jakarta.persistence. Table ;

@Entity
@Table (name = "accounts" )
public class Account extends BaseEntity {

    @Column (nullable = false , unique = true )
    private String accountNumber ;

    @Column (nullable = false )
    private Double balance ;

    public Account () {
    }
}
```

```
public Account (String accountNumber, Double balance) {  
    this . accountNumber = accountNumber;  
    this.balance = balance ;  
}  
  
public String getAccountNumber () {  
    return accountNumber ;  
}  
  
public void setAccountNumber (String accountNumber) {  
    this . accountNumber = accountNumber;  
}  
  
public Double getBalance () {  
    return balance ;  
}  
  
public void setBalance (Double balance) {  
    this.balance = balance ;  
}  
}
```

3. Repository, service and controller

Create :

- **AccountRepository** extends **JpaRepository<Account, Long>**;
- **AccountService** with **createAccount()** and **getAccounts()**;
- **AccountController** with:
 - **POST /api/accounts**
 - **GET /api/accounts**

5.4. digibank-transaction module

1. Additions

Create this module using the same principle

2. Transaction Entity

```
package com.digibank.transaction.model;

import com.digibank.shared.model.BaseEntity;
import jakarta.persistence. Column ;
import jakarta.persistence. Entity ;
import jakarta.persistence. Table ;

@Entity
@Table (name = "transactions" )
public class Transaction extends BaseEntity {

    @Column (nullable = false )
```

```
private String type ;

@Column (nullable = false )

private Double amount ;

public Transaction () {

}

public Transaction (String type, Double amount) {

    this.type = type ;

    this.amount = amount ;

}

public String getType () {

    return type ;

}

public void setType (String type) {

    this.type = type ;

}

public Double getAmount () {

    return amount ;

}
```

```
public void setAmount (Double amount) {  
    this.amount = amount ;  
}  
}
```

4. Repository, service and controller

Create :

- **TransactionRepository** extends **JpaRepository<Transaction, Long>**;
- **TransactionService** with **addTransaction()** and **getTransactions()**;
- **TransactionController** with:
 - **POST /api/transactions**
 - **GET /api/transactions**

5.5. Digibank Compliance Module

1. Additions

This module can intentionally remain lighter in this workshop, in order to maintain a feasible scope while respecting the Digi Bank structure.

2. Compliance Service

```
package com.digibank.compliance.service;  
  
import org.springframework.stereotype. Service ;
```

```

@Service

public class ComplianceService {

    public boolean validateTransactionAmount (Double amount) {

        return amount != null && amount <= 10000 ;

    }

}

```

3. REST Controller ComplianceController

```

package com.digibank.compliance.api;

import com.digibank.compliance.service.ComplianceService;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping ;
import org.springframework.web.bind.annotation.PathVariable ;
import org.springframework.web.bind.annotation.RequestMapping ;
import org.springframework.web.bind.annotation.RestController ;

import java.util.Map;

@RestController

@RequestMapping ( "/api/compliance" )

```

```

public class ComplianceController {

    private final ComplianceService service ;

    public ComplianceController (ComplianceService service) {

        this.service = service ;

    }

    @GetMapping ( "/validate/{amount}" )

    public ResponseEntity<Map<String, Boolean>> validate (
@PathVariable Double amount) {

        return ResponseEntity. ok (Map. of ( "valid" , service
.validateTransactionAmount(amount))) ;

    }

}

```

5.5. digibank-index index module

1. pom.xml

```

<project xmlns = "http://maven.apache.org/POM/4.0.0"
    xmlns: xsi = "http://www.w3.org/2001/XMLSchema-instance"
    xsi :schemaLocation = "http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd

```

```
<parent>

<groupId> com.digibank </groupId>

<artifactId> digibank-parent-spring </artifactId>

<version> 1.0.0 </version>

</parent>

<modelVersion> 4.0.0 </modelVersion>

<artifactId> digibank-index </artifactId>

<packaging> jar </packaging>

<dependencies>

<dependency>

<groupId> com.digibank </groupId>

<artifactId> digibank-shared </artifactId>

<version> 1.0.0 </version>

</dependency>

<dependency>

<groupId> org.springframework.boot </groupId>

<artifactId> spring-boot-starter-web </artifactId>

</dependency>

<dependency>

<groupId> org.springframework.boot </groupId>

<artifactId> spring-boot-starter-thymeleaf </artifactId>

</dependency>
```

```
</dependencies>
```

```
</project>
```

2. Index controller

```
package com.digibank.index.web;

import org.springframework.stereotype. Controller ;
import org.springframework.web.bind.annotation. GetMapping ;

@Controller
public class IndexController {

    @GetMapping ( "/" )
    public String index () {
        return "index" ;
    }
}
```

3. View index.html

Create src/main/resources/templates/index.html with a simple page showcasing Digi Bank, its modules, and the main REST endpoints. This page will serve the same minimal showcase role as in Workshop 2.

```
<!DOCTYPE html >

<html lang ="en" >

<head>

<meta charset ="UTF-8" >

<meta name ="viewport" content ="width=device-width,
initial-scale=1.0" >

<title> Digi Bank - Home </title>

<style>

    * {

margin: 0;

padding: 0;

box-sizing: border-box;

}

body {

font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;

background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);

min-height: 100vh;

display: flex;

justify-content: center;
```

```
align-items: center;

padding: 20px;

}

.container {

background: white;

border-radius: 10px;

box-shadow: 0 10px 40px rgba(0, 0, 0, 0.2);

max-width: 900px;

width: 100%;

padding: 50px;

text-align: center;

}

.logo {

font-size: 48px;

margin-bottom: 20px;

color: #667eea;

}

h1 {

color: #333;

margin-bottom: 20px;

font-size: 36px;
```

```
}

.version {
color: #888;
font-size: 14px;
margin-bottom: 30px;
}

.description {
color: #555;
font-size: 16px;
line-height: 1.8;
margin: 30px 0;
text-align: justify;
}

.features {
background: #f5f5f5;
padding: 30px;
border-radius: 8px;
margin: 30px 0;
text-align: left;
}
```

```
.features h3 {
color: #667eea;
margin-bottom: 15px;
}

.features ul {
list-style: none;
padding: 0;
}

.features li {
padding: 8px 0;
color: #555;
border-bottom: 1px solid #ddd;
}

.features li:last-child {
border-bottom: none;
}

.features li:before {
content: "✓ ";
color: #667eea;
font-weight: bold;
```

```
margin-right: 10px;
}

.description h2 {
color: #667eea;
margin-top: 20px;
margin-bottom: 15px;
font-size: 20px;
}

.description p {
margin-bottom: 15px;
}

.section-box {
background: #f8f9fa;
padding: 25px;
border-radius: 8px;
margin: 25px 0;
border-left: 4px solid #667eea;
}

.section-box h3 {
color: #667eea;
```

```
margin-bottom: 15px;

font-size: 18px;

}

.section-box.highlighted {
background: linear-gradient(135deg, #f0f4ff 0%, #f8f9ff 100%);
border-left-color: #764ba2;
}

.functional-list, .requirements-list {
list-style: none;
padding: 0;
}

.functional-list li, .requirements-list li {
padding: 12px 0;
color: #555;
border-bottom: 1px solid #e0e0e0;
font-size: 15px;
line-height: 1.6;
}

.functional-list li:last-child, .requirements-list li:last-child
{
```

```
border-bottom: none;

}

.requirements-list li {
font-size: 14px;
}

.about-section {
background: #f5f5f5;
padding: 25px;
border-radius: 8px;
margin: 30px 0;
border-top: 3px solid #667eea;
}

.about-section h3 {
color: #667eea;
margin-bottom: 15px;
font-size: 18px;
}

.about-section p {
color: #555;
font-size: 14px;
```

```
line-height: 1.8;

margin-bottom: 10px;

}

.links {

margin: 40px 0 0 0;

display: flex;

gap: 15px;

justify-content: center;

flex-wrap: wrap;

}

.btn {

display: inline-block;

padding: 12px 30px;

border-radius: 5px;

text-decoration: none;

font-weight: 600;

transition: all 0.3s ease;

cursor: pointer;

border: none;

font-size: 14px;

}
```

```
.btn-primary {  
background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);  
color: white;  
}  
  
.btn-primary:hover {  
transform: translateY(-2px);  
box-shadow: 0 5px 20px rgba(102, 126, 234, 0.4);  
}  
  
.btn-secondary {  
background: white;  
color: #667eea;  
border: 2px solid #667eea;  
}  
  
.btn-secondary:hover {  
background: #f5f5f5;  
transform: translateY(-2px);  
}  
  
.api-info {  
background: #e8f4f8;  
padding: 20px;
```

```
border-left: 4px solid #667eea;

margin: 30px 0;

text-align: left;

border-radius: 5px;
}

.api-info h4 {

color: #667eea;

margin-bottom: 10px;
}

.api-info p {

color: #555;

font-size: 14px;

font-family: 'Courier New', monospace;

margin: 5px 0;
}

footer {

margin-top: 40px;

padding-top: 30px;

border-top: 1px solid #eee;

color: #888;

font-size: 12px;
```

```

}

    </style>

</head>

<body>

<div class ="container" >

<div class ="logo" >  </div>

<h1> Digi Bank </h1>

<div class ="version" > Version 1.0 - Workshop 2 </div>

<div class ="description" >

<h2> Overview </h2>

<p>

<strong> Digi Bank </strong> is a capstone project that
implements the central banking system

of a new digital bank. This project serves as a shared
foundation to gradually build

a coherent vision of cloud architecture applied to a real,
critical, and scalable system.

        </p>

<p>

                Digi Bank is a digital bank in its launch phase
aiming to offer customers

a fully digital banking service with a smooth, secure, and
continuously available experience.

To achieve this, it must deploy a <strong> Digital Core Banking
System </strong> – a centralized

```

platform able to handle essential banking operations.

</p>

</div>

<div class ="section-box" >

<h3> Core Functions </h3>

<ul class ="functional-list" >

 Customer Management : creating and updating profiles, personal information, status, and history

 Account Management : opening, viewing, updating and closing bank accounts

 Transaction Management : transfers, deposits, withdrawals, payments and history viewing

 Credit and Loans : recording applications, tracking installments and lifecycle management

 Compliance and Security : KYC, AML, access control and traceability

 Open Banking : exposure controlled via APIs to external partners

</div>

<div class ="section-box highlighted" >

<h3> Requirements & Constraints </h3>

<ul class ="requirements-list" >

```

<li>  <strong> High Availability </strong> : essential
functions always accessible </li>

<li>  <strong> Strong Security </strong> : security by design,
protection of sensitive data </li>

<li>  <strong> Scalability </strong> : support for growing
numbers of customers and operations </li>

<li>  <strong> Transaction Reliability </strong> : consistency
and traceability of critical operations </li>

<li>  <strong> Resilience </strong> : limiting the impact of
partial failures </li>

<li>  <strong> Compliance </strong> : meeting regulatory
requirements (KYC, AML, GDPR) </li>

<li>  <strong> Openness & Integration </strong> :
well-controlled APIs for external partners </li>

</ul>

</div>

<div class ="features" >

<h3> Modular Architecture (Workshop 2) </h3>

<ul>

<li> Shared Module: common entities and utilities </li>

<li> Customer Module: customer profile management </li>

<li> Account Module: bank account management </li>

<li> Transaction Module: management of financial operations
</li>

<li> Compliance Module: checks and regulatory compliance </li>

<li> Index Module: homepage and documentation </li>


```

```

</ul>

</div>

<div class ="api-info" >

<h4> 📡 REST API Access </h4>

<p><strong> Base URL: </strong>
http://localhost:8080/digibank-app/api/ </p>

<p><strong> Available endpoints: </strong></p>

<p> • GET /api/index - General information </p>

<p> • POST /api/customers - Create a customer </p>

<p> • GET /api/customers - List customers </p>

<p> • POST /api/accounts - Create an account </p>

<p> • GET /api/accounts - List accounts </p>

<p> • POST /api/transactions - Create a transaction </p>

<p> • GET /api/transactions - List transactions </p>

<p> • GET /api/compliance/validate/{amount} - Validate an amount
</p>

</div>

<div class ="links" >

<a href ="http://localhost:8080/digibank-app/api/" class ="btn
btn-primary" > 📡 REST API </a>

<a href ="/docs" class ="btn btn-secondary" > 📖 Documentation
</a>

</div>

```

```
<div class ="about-section" >

<h3> Capstone Project </h3>

<p>

        Digi Bank is an educational project on cloud
architecture applied to a real banking system.

It covers progressive learning of monolithic architecture,
microservices,

high availability, resilience, scalability and security.

        </p>

<p>

<strong> Course Instructor: </strong> Eng. Willy Damtchou <br>

<strong> Degree: </strong> Professional Degree in Cloud
Computing - UDM <br>

<strong> Period: </strong> June 2026

        </p>

</div>


<footer>

<p><strong> Digi Bank - Digital Core Banking System
</strong></p>

<p> Practical Work - Capstone Project </p>

<p> Professional Degree in Cloud Computing - University of the
Mountains </p>

<p> Instructor: Eng. Willy Damtchou | June 2026 </p>

</footer>
```

```
</div>

</body>

</html>
```

6. Spring Boot application module

6.1. Main Class

Create the following main class in digibank-app:

```
package com.digibank.app;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.boot.builder.SpringApplicationBuilder;
import org.springframework.boot.web.servlet.support.SpringBootServletInitializer;

@SpringBootApplication
public class DigiBankApplication extends SpringBootServletInitializer {
```

```

    public static void main (String[] args) {
        SpringApplication.run (DigiBankApplication.class , args);
    }

    @Override
    protected SpringApplicationBuilder configure
(SpringApplicationBuilder builder) {
        return builder.sources ( DigiBankApplication.class );
    }
}

```

6.2. pom.xml of the application module

Create **digibank-app/pom.xml** with:

```

<project xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd"

    <parent>

    <groupId> com.digibank </groupId>

```

```
<artifactId> digibank-parent-spring </artifactId>

<version> 1.0.0 </version>

</parent>

<modelVersion> 4.0.0 </modelVersion>

<artifactId> digibank-app </artifactId>

<packaging> war </packaging>

<dependencies>

<dependency>

<groupId> com.digibank </groupId>

<artifactId> digibank-shared </artifactId>

<version> 1.0.0 </version>

</dependency>

<dependency>

<groupId> com.digibank </groupId>

<artifactId> digibank-customer </artifactId>

<version> 1.0.0 </version>

</dependency>

<dependency>

<groupId> com.digibank </groupId>

<artifactId> digibank-account </artifactId>

<version> 1.0.0 </version>

</dependency>
```

```
<dependency>

<groupId> com.digibank </groupId>

<artifactId> digibank-transaction </artifactId>

<version> 1.0.0 </version>

</dependency>

<dependency>

<groupId> com.digibank </groupId>

<artifactId> digibank-compliance </artifactId>

<version> 1.0.0 </version>

</dependency>

<dependency>

<groupId> com.digibank </groupId>

<artifactId> digibank-index </artifactId>

<version> 1.0.0 </version>

</dependency>

<dependency>

<groupId> org.springframework.boot </groupId>

<artifactId> spring-boot-starter-web </artifactId>

<exclusions>

<exclusion>

<groupId> org.springframework.boot </groupId>

<artifactId> spring-boot-starter-tomcat </artifactId>

</exclusion>

</exclusions>
```

```
</dependency>

<dependency>

<groupId> org.springframework.boot </groupId>

<artifactId> spring-boot-starter-logging </artifactId>

<exclusions>

<exclusion>

<groupId> ch.qos.logback </groupId>

<artifactId> logback-classic </artifactId>

</exclusion>

<exclusion>

<groupId> org.apache.logging.log4j </groupId>

<artifactId> log4j-to-slf4j </artifactId>

</exclusion>

</exclusions>

</dependency>

<dependency>

<groupId> org.slf4j </groupId>

<artifactId> slf4j-api </artifactId>

<scope> provided </scope>

</dependency>

<dependency>

<groupId> jakarta.servlet </groupId>

<artifactId> jakarta.servlet-api </artifactId>

<scope> provided </scope>
```

```
</dependency>

</dependencies>

<build>

<finalName> digibank-app </finalName>

<plugins>

<plugin>

<groupId> org.springframework.boot </groupId>

<artifactId> spring-boot-maven-plugin </artifactId>

</plugin>

<plugin>

<groupId> org.apache.maven.plugins </groupId>

<artifactId> maven-war-plugin </artifactId>

</plugin>

<plugin>

<groupId> org.wildfly.plugins </groupId>

<artifactId> wildfly-maven-plugin </artifactId>

<configuration>

<hostname> localhost </hostname>

<port> 9990 </port>

<username> admin </username>

<password> admin </password>

</configuration>

</plugin>
```

```
</plugins>

</build>

</project>
```

6.3. application.properties file

Create **digibank-app/src/main/resources/application.properties** . Two approaches are possible:

- either the application connects directly to PostgreSQL using `spring.datasource.url`;
- It can either rely on a data source declared in WildFly, which is more consistent with the chosen deployment method. This workshop will prioritize continuity with WildFly. However, students can start with a simple local configuration to expedite initial testing.

```
spring.datasource.jndi-name = java:/jdbc/DigiBankDS

spring.datasource.driver-class-name = org.postgresql.Driver

spring.jpa.hibernate.ddl-auto = update

spring.jpa.database-platform = \
org.hibernate.dialect.PostgreSQLDialect

logging.level.root = INFO
```

7. WildFly configuration

7.1. Traditional Deployment in WAR

The application must be compiled as a WAR file and then deployed to WildFly. This method is consistent with a Spring Boot application configured for an external container using `SpringBootServletInitializer`.

7.2. PostgreSQL Driver

Download the PostgreSQL driver and install it as a WildFly module if necessary. The general principle is to provide the driver to the server so that it can expose a PostgreSQL datasource.

7.3. Declaration of the data source

Declare a datasource `java:/jdbc/DigiBankDS` in WildFly, according to the logic already used in workshop 2.

Example parameters:

- JNDI Name: `java:/jdbc/DigiBankDS`
- Database: `jdbc:postgresql://localhost:5432/digibankdb`
- User: `digibankuser`
- Password: `digibankpwd`

7.4. Expected Result

WildFly must start without error, recognize the PostgreSQL datasource and be ready to receive the WAR from the Digi Bank application.

8. Compilation, packaging and deployment

8.1. Complete Compilation

From the project's origin:

```
mvn clean install
```

8.2. WAR Generation

```
mvn -pl digibank-app clean package
```

8.3. Expected Result

digibank-app.war file must be generated in the target folder of the application module.

8.4. Deployment on WildFly

The deployment must be configured via Maven.

8.5. Expected result

The application must be deployed and accessible via the local server URL

http://ip:port_wildfly/digibank-app

9. Functional checks

Test the following endpoints:

- GET /api/customers
- POST /api/customers

- GET /api/accounts
- POST /api/accounts
- GET /api/transactions
- POST /api/transactions
- GET /api/compliance/validate/5000
- GET /api/compliance/validate/25000

Example curl command to create a client:

```
curl -X POST http://localhost:8080/digibank-app/api/customers \
-H "Content-Type: application/json" \
-d '{"firstName":"Ali","lastName":"Diallo","email":"ali.diallo@digibank.com"}'
```

10. JUnit Unit Tests

10.1. Test Dependencies

Add the Spring Boot and JUnit 5 test dependencies to the relevant modules.

10.2. Example ComplianceServiceTest

```
package com.digibank.compliance.service;

import org.junit.jupiter.api.Assertions;
import org.junit.jupiter.api. Test ;
```

```
public class ComplianceServiceTest {

    private final ComplianceService service = new ComplianceService();

    @Test

    void shouldAcceptAmountBelowLimit () {
Assertions. assertTrue ( service.validateTransactionAmount ( 5000.0
));
    }

    @Test

    void shouldRejectAmountAboveLimit () {
Assertions. assertFalse ( service.validateTransactionAmount ( 20000.0
));
    }
}
```

10.3. Test execution

```
mvn test
```

10.4. Expected Result

The tests must run without error and confirm the expected behavior of the compliance service.

11. Database testing with Cucumber

11.1. Maven Dependencies

Add the necessary dependencies for Cucumber to the relevant module.

11.2. Feature File

Create src/test/resources/features/compliance.feature:

```
Feature : Transaction amount validation

Scenario : Amount as agreed
    Given a transaction amount of 5000
    When the compliance check is performed
    Then the transaction is accepted

Scenario : Incorrect amount
    Given a transaction amount of 20,000
    When the compliance check is performed
    Then the transaction is rejected
```

11.3. Definition of steps

```
package com.digibank.compliance.steps;

import com.digibank.compliance.service.ComplianceService;
import io.cucumber.java.fr. Then ;
import io.cucumber.java.fr. Etantdonnée ;
import io.cucumber.java.fr. When ;
import org.junit.jupiter.api.Assertions;

public class ComplianceSteps {

    private final ComplianceService service = new ComplianceService();
    private Double amount ;
    private boolean result ;

    @GivenAs ( "a transaction amount of {int}" )
    public void aTransactionAmountOf (Integer amount) {
        this . amount = amount.doubleValue();
    }

    @When ( "the compliance check is executed" )
    public void ComplianceCheckIsExecute () {
        result = service.validateTransactionAmount ( amount );
    }
}
```

```

}

    @So ( "the transaction is accepted" )

    public void theTransactionIsAccepted () {
Assertions. assertTrue ( result );
    }

    @So ( "the transaction is rejected" )

    public void theTransactionIsRejected () {
Assertions. assertFalse ( result );
    }
}

```

11.4. Runner Cucumber

```

package com.digibank.compliance;

import org.junit.platform.suite.api. IncludeEngines ;
import org.junit.platform.suite.api. SelectClasspathResource ;
import org.junit.platform.suite.api. Following ;

@Following
@IncludeEngines ( "cucumber" )
@SelectClasspathResource ( "features" )

```

```
public class RunCucumberTest {  
  
}
```

11.4. Expected Result

Cucumber scenarios should allow verification of business behavior in a form close to functional requirements expressed in natural language.

12. Work assigned to students

Students must complete all of the following tasks:

1. Install the complete development environment.
2. Configure IntelliJ IDEA or VS Code.
3. Create the PostgreSQL database and the application user.
4. Build the multi-module Maven structure.
5. Create the set of pom.xml files.
6. Develop the shared, customer, account, transaction, compliance, index and app modules.
7. Configure Spring Boot, JPA, PostgreSQL and WAR packaging.
8. Prepare the deployment on WildFly.
9. Compile, package and deploy the application.
10. Test the REST endpoints.
11. Implement JUnit tests.
12. Implement the Cucumber scenarios.
13. Correct any errors until a stable executable version is obtained.

13. Expected deliverables



Students must submit:

1. The complete digibank-parent-spring project
2. All Maven modules created
3. Configuration files
4. SQL scripts for database creation
5. Spring Boot Java classes
6. REST controllers
7. Services and repositories
8. JUnit tests
9. The Cucumber scenarios
10. A short note explaining the order of development followed and the main difficulties encountered.

14. Evaluation criteria

The evaluation will be based on the following criteria:

1. Maven tree structure consistency
2. accuracy of technical configurations
3. good modular cutting
4. quality of the Spring Boot code produced
5. validity of REST controllers
6. how persistence works
7. correct execution under WildFly
8. quality and relevance of JUnit tests
9. quality and relevance of Cucumber scenarios
10. respect for the logical order and method of execution.

15. Expected results



At the end of the workshop, the student should have a first executable version of Digi Bank, built around a modular monolith in Spring Boot, including the essential functional areas, a connection to PostgreSQL, basic business services, operational REST endpoints, a WAR package compatible with WildFly and a first level of technical and functional tests.

This step directly prepares the way for the next phase of the overarching project. It paves the way for subsequent work focusing on enriching the business domain, securing the application, improving transactional robustness, technical documentation, and preparing for a potential transition to a more distributed architecture.